

SPEXTc

Plataforma Acadêmica de Visão Computacional e Tokenização de Imagens

Documento de Contexto Completo

Gustavo Barros · FATEC · 2026

Horizonte: 6 meses – 2 anos

Este documento resume a visão, a arquitetura, as decisões fechadas, o inventário de modelos e o roadmap do projeto **SPEXTc**. Serve como referência completa de contexto para colaboradores, orientadores e alunos de TCC.

1. O que é o SPEXTc

O **SPEXTc** é uma **plataforma acadêmica de percepção visual** onde a unidade fundamental não é a imagem — é o **token de imagem**. Toda imagem que entra no sistema é convertida em um conjunto padronizado de tokens, e tudo o que vem depois (detecção, classificação, navegação, relatório, busca) são *operações sobre tokens*.

FRASE-GUIA

"Automatizar o automatizado. Sistemas que já agem por regras passam a consumir o SPEXTc como camada de percepção adaptativa — e cada uso real vira dado que melhora o próximo modelo."

O SPEXTc NÃO é (ainda):

- Um SaaS comercial fechado.
- Uma única API de YOLO.
- Um sistema industrial com esteira/NIR/soprador.

É uma **base reutilizável** onde cada projeto pluga um **domínio** (vocabulário + pipeline + regras de negócio) sem reescrever o núcleo. Cada TCC que entra melhora a plataforma para todos — um flywheel acadêmico.

Flywheel de aprendizado

uso → dados → modelo melhor → uso melhor → mais dados → ...

O carro autônomo de brinquedo (competição SENAC) é o caso mais ilustrativo: o próprio desempenho do carro (bateu? completou a volta?) é **ground truth implícito** que retreina o modelo — sem anotação humana para tudo.

2. Os três primeiros domínios

Insight central: **Baja e Soja usam paradigmas opostos** (detecção vs. segmentação+classificação) e mesmo assim produzem o **mesmo TokenSet**. Isso prova que o core precisa suportar múltiplos pipelines atrás de um contrato único — nunca assumir 'é tudo YOLO'.

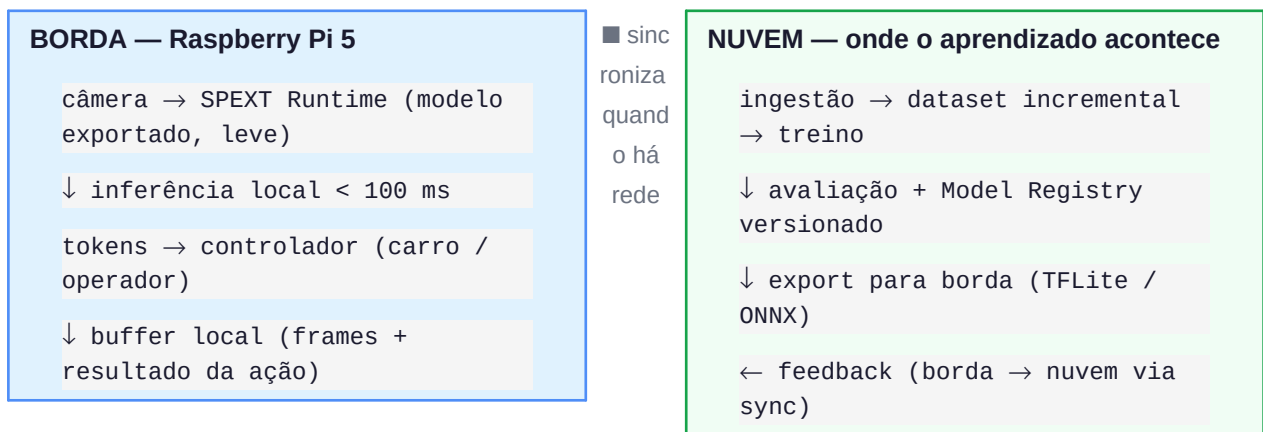
Domínio	Pipeline	Regime	Feedback	Estado
Baja (pintura)	YOLO (detecção nativa)	Batch / nuvem	Humano confirma	Protótipo demo (pesos COCO)
Soja (grãos)	OpenCV recorta → EfficientNet-B0	Batch / nuvem	Humano confirma lote	Modelos treinados (Drive, .keras)
Carro SENAC (autônomo)	Leve, real-time (a definir)	Borda Pi 5, < 100 ms	Automático (desempenho)	Rascunho

Como um TCC vira um domínio

1. Define o vocabulário de classes. 2. Escolhe um pipeline existente (ou propõe um novo — isso já é contribuição). 3. Traz o dataset versionado. 4. Treina pela infraestrutura do SPEXTc; modelo entra no registry. 5. Registra o domínio. Upload, overlay, histórico, API e learning loop funcionam de graça.

3. Arquitetura — duas metades

A divisão em duas metades resolve a tensão entre domínios em batch (Soja/Baja — precisão importa, latência não) e o domínio em tempo real (Carro SENAC — latência < 100 ms é lei).



Regra de ouro (nunca violar):

A borda nunca treina, só infere e coleta.

A nuvem nunca infere em tempo real, só treina e versiona.

As três camadas do core

Camada	Responsabilidade	Estado
1. Perception Core	imagem → tokens (agnóstico de domínio)	Código na Fase 1 ■
2. Domain Registry	vocabulário + pipeline + regras por domínio	Código na Fase 1 ■
3. Learning Loop	coleta → retreino → versiona → re-deploy	Esqueleto (Fase 5)

4. O contrato de Token

O **Token** é a interface do SPEXTc. Não é uma feature — é o que faz Baja, Soja e Carro serem o mesmo sistema. Um defeito de pintura, um grão ardido e um obstáculo na pista são todos:

```
Token { classe, dominio, bbox[x1,y1,x2,y2], confianca, mask?, embedding?,  
semantica{decisao, ...} }
```

O agregado de uma imagem é o **TokenSet**, que carrega a **versão exata do modelo** que o gerou — essencial para rastreabilidade e reprodutibilidade científica.

Pipelines de tokenização aceitos pelo core:

Pipeline	Como tokeniza	Quem usa hoje
Detecção (YOLO/RT-DETR)	Rede devolve caixas + classe diretamente	Baja ■
Seg + Classificação (OpenCV + EfficientNet)	OpenCV recorta cada objeto → classificador rotula → bbox vem do recorte	Soja ■
Patch/Embedding (ViT)	Imagem fatiada em patches → embeddings	Futuro / TCC
Codebook discreto (VQ-VAE)	Vocabulário visual aprendido	Futuro / pesquisa

A plataforma começa em **Nível 1 (tokens de objeto)** — já viável com Baja e Soja — sem fechar a porta para Nível 2 (patches ViT) ou Nível 3 (tokens discretos), linhas de pesquisa futuras.

5. Borda — Raspberry Pi 5

Hardware de borda disponível hoje: **apenas o Raspberry Pi 5**. É o cérebro de inferência local do domínio Carro SENAC.

Modelo	Cabe no Pi 5?	Observação
EfficientNet-B0 (29 MB, Keras→TFLite)	■ Sim	Poucos a dezenas de FPS conforme quantização
YOLO nano (ONNX/TFLite)	■ Sim	Viável para detecção leve em tempo real
RT-DETR / modelos grandes	■■ Marginal	Só com folga de latência; não recomendado para o carro
EfficientNet-B0 no ESP32	■ Não	29 MB; ESP32 tem ~520 KB RAM. Não cabe.

Papel de cada peça (arquitetura típica de competição):

```

ESP32-CAM → captura frame
↓ (USB / WiFi local)
Raspberry Pi 5 → inferência (SPEXT Runtime) → token → decisão
↓
ESP32 / driver → aciona motor / direção

```

Enquanto não há ESP32 em mãos, a câmera conecta direto ao Pi. Quando o ESP32 chegar, entra como olho (captura) e mão (atuação) — o Pi continua sendo o cérebro.

Linha de pesquisa / TCC: destilar um modelo pequeno o suficiente para rodar no ESP32 (de 29 MB para < 1 MB via quantização, pruning e knowledge distillation) é um TCC inteiro de model compression.

6. Learning Loop — melhoria progressiva

Dataset e modelos **melhoram progressivamente** — por decisão explícita do dono do projeto. Isso é **fundação, não fase futura**. Versionamento e rastreabilidade existem desde o primeiro commit.

O loop completo:

```
Inferência em produção
↓ sinal de qualidade
■■■ automático: desempenho do carro → amostra auto-rotulada
■■■ humano: confirma/corrigir → amostra revisada
↓
Dataset incremental versionado
↓ retreino agendado
Avaliação vs. modelo atual
■■■ melhor → Model Registry (nova versão) → A/B → deploy → re-export
■■■ pior → descarta candidato
```

As duas fontes de feedback:

Fonte	Domínios	Como funciona
Humano	Baja, Soja	Operador confirma ou corrige os tokens; correções viram rótulos de alta qualidade.
Automático	Carro SENAC	Resultado da ação (bateu/completou volta/tempo) é ground truth implícito — sem anotação manual.

O que é versionado desde o dia 1:

- **Datasets:** cada versão é imutável e identificável (ex.: soja-ds@2026.06). Novas amostras entram como incremento, nunca sobrescrevendo.
- **Modelos (Model Registry):** cada peso carrega metadados (dataset de origem, hiperparâmetros, métricas, data). O TokenSet carrega a versão exata que o gerou — qualquer resultado é reproduzível.
- **Regra de promoção:** nunca troca às cegas. Candidato só substitui o modelo de produção se for melhor nas métricas que importam ao domínio.

7. Inventário de modelos e datasets

Retrato do que já existe e funciona. A fonte da verdade será o Model Registry quando o código estiver completo.

Modelos da Soja — Google Drive (todos EfficientNet-B0, .keras):

Arquivo	Tamanho	Papel
soja_phase1_best.keras	~17 MB	Transfer learning inicial
soja_phase2_best.keras	~29 MB	Melhor da fase 2 (fine-tuning)
soja_model_final.keras	~29 MB	Modelo final
soja_finetuned_best.keras	~29 MB	Melhor do fine-tuning com fotos próprias
soja_finetuned_final.keras	~29 MB	Fine-tuned final (em uso)

Dataset da Soja:

SoyaBeans Classifications v2 (Roboflow, MIT). ~12.528 imagens, 400×400 px → resize 224×224. 5 classes: Broken · Immature · Intact · Skin-damaged · Spotted. Splits: train/valid/test. **Evolução prevista:** coleta própria (fotos de celular, fundo escuro) para mitigar domain shift + fine-tuning progressivo.

Modelo e dataset da Baja:

Modelo atual: yolov8n.pt (COCO) em modo demo — sem fine-tune com defeitos reais ainda. **Dataset:** a construir (meta 500–1000 imagens de chassis pintados, rotulação Roboflow/CVAT, formato YOLO txt). Em construção progressiva.

Os dois paradigmas lado a lado:

	Baja	Soja
Pipeline	Detecção (YOLO)	Segmentação OpenCV + Classificação EfficientNet
Caixa vem de	A própria rede	Recorte do OpenCV
Formato do peso	.pt (Ultralytics)	.keras (TensorFlow/Keras)
Export para borda	ONNX	TFLite
Estado	Demo (sem fine-tune)	Treinado e funcionando ■

8. Decisões fechadas — não relitigar

Estas decisões foram tomadas com contexto, têm razões documentadas e **não devem ser reabertas** sem justificativa forte.

Tema	Decisão	Por quê
Contrato central	Token / TokenSet	Unifica domínios e as duas metades. Interface, não feature.
Hardware de borda	Raspberry Pi 5 (único hoje)	Roda EfficientNet-B0 / YOLO nano via TFLite/ONNX < 100 ms.
ESP32	Sensor/atuador, NÃO inferência	29 MB de modelo não cabe em 520 KB RAM. ESP32-CAM captura, Pi infere.
Treino	Nuvem (Colab/GPU), nunca na borda	Borda só infere e coleta. Regra de ouro.
Modelo Soja	EfficientNet-B0 (Keras) já treinado	É o que existe e funciona. O data.yaml YOLO era aspiracional.
Modelo Baja	YOLO (Ultralytics) em modo demo	Protótipo existente. Fine-tune após coletar dataset.
Versionamento	Model Registry + datasets versionados desde já	Melhoria é progressiva — é fundação, não fase futura.

Tema	Decisão	Por quê
Deploy nuvem	HF Spaces + Vercel	O dono já usa. Evitar retrabalho de infraestrutura.
Core	Múltiplos pipelines atrás do mesmo contrato	Baja (detecção) e Soja (seg+classif) provam que é necessário.

9. Estado do código (Fase 1 completa)

A Fase 1 (fundação do core) foi implementada e testada — **21 testes passando**. A estrutura abaixo está viva no repo Gustavo-dev04/SPEXTc, branch claude/loving-cannon-w66dmi.

```

spxt/
  core/
    tokens.py          # Perception Core – agnóstico de domínio
    versioning.py      # Token / TokenSet (contrato central)
    pipeline.py        # ModelRef "nome@versao"
    pipelines.py       # PipelineKind + Pipeline (ABC)
    domain.py          # Detection / SegClassif + StubPipeline
    registry.py        # Domain declarativo
    perception.py      # DomainRegistry
    domains/           # PerceptionCore
    baja.py · soja.py · carro_senac.py
  edge/runtime.py     # Domínios plugáveis
  cloud/              # Esqueleto runtime Pi 5
  tests/              # Esqueleto metade de nuvem
                    # 21 testes (token, registry, perception)

```

O que já está provado em código:

O que	Como está provado
Token é real	Validação Pydantic: bbox coerente, confiança 0–1, extras proibidos
Dois paradigmas convivem	DetectionPipeline e SegClassifPipeline atrás do mesmo TokenSet
Domínio é declarativo	Baja, Soja e Carro registrados; TCC = módulo + registro
Wiring ponta a ponta	test_perception: imagem→tokens→thresholds→decisão via StubPipeline
Versionamento	ModelRef valida 'nome@versao'; TokenSet carrega a versão exata

10. Roadmap

Horizonte: **6 meses a 2 anos**. Primeiro marco prático: competição de carros autônomos no SENAC.

Fase	Entregas	Status
0 — Blueprint	Visão, arquitetura, domínios, borda, learning loop documentados no repo	■ Concluída

Fase	Entregas	Status
1 — Fundação do core	Token/TokenSet (Pydantic), Domain Registry, Pipelines (esqueletos + Stub), 21 testes	■ Concluída
2 — Portar domínios	Soja plugável (EfficientNet .keras → TokenSet) Baja plugável (YOLO demo → TokenSet) Frontend único com overlay para qualquer domínio	■ Próxima
3 — Borda Pi 5	Export Keras→TFLite e YOLO→ONNX quantizados SPEXT Runtime: modelo exportado → TokenSet Buffer local resiliente a offline	
4 — Carro SENAC	Vocabulário e pipeline real-time definidos Feedback automático fechado (desempenho = treino)	
5 — Learning Loop	Ingestão → retreino → A/B → re-export Volta completa do flywheel demonstrada	
6 — Plataforma / TCCs	Onboarding de novos domínios por terceiros API documentada, dashboard multi-domínio Deploy: HF Spaces + Vercel	

Linhas de pesquisa / TCC (paralelas):

- **Tokenização avançada:** subir de tokens de objeto para patches (ViT) ou tokens discretos (VQ-VAE).
- **Model compression:** destilar EfficientNet/YOLO para < 1 MB rodando no ESP32.
- **Aprendizado contínuo:** feedback de operação fechada como objeto de estudo e publicação.
- **NIR/multiespectral:** defeitos internos invisíveis ao RGB (horizonte, herdado da Soja).

11. Repositórios relacionados

Repositório	O que é	Estado
Gustavo-dev04/SPEXTc	Plataforma central (este projeto)	Em desenvolvimento ■
Gustavo-dev04/Baja	Inspeção de pintura (YOLO + FastAPI + Next.js)	Protótipo demo
Gustavo-dev04/soja-inspectio-(branch: claude/soja-inspection-setup-b2jaG)	Inspeção de soja (EfficientNet + FastAPI + Next.js + Supabase)	Modelos treinados ■

Nota sobre o nome

O repositório foi renomeado de SPEXT para SPEXTc no GitHub. O redirect está ativo, mas o remote local ainda aponta para o nome antigo. Rodar `git remote set-url origin https://github.com/Gustavo-dev04/SPEXTc.git` quando conveniente.